Subject: ASD&D **Semester:** V

Class: TE **Batch:** T11

Name: Ananya Amesur **Roll No:** 2401005

Experiment 7

Aim: To implement monitoring using Prometheus / Grafana / Nagios

Lab Outcome: LO5

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring is a process to detect, report, and respond to all the attacks which occur in its infrastructure. Once the application is deployed into the server, the role of continuous monitoring comes into play. The entire process is all about taking care of the company's infrastructure and responding appropriately.

Why Do We Need Nagios Tools?

Here, are the important reasons to use Nagios monitoring tool:

- Detects all types of network or server issues
- Helps you to find the root cause of the problem which allows you to get the permanent solution to the problem
- Active monitoring of your entire infrastructure and business processes
- Allows you to monitor and troubleshoot server performance issues
- Helps you to plan for infrastructure upgrades before outdated systems create failures
- You can maintain the security and availability of the service
- Automatically fix problems in a panic situation
- Nagios is the most popular, open source, powerful monitoring system for any kind of infrastructure. It enables organizations to identify and resolve IT infrastructure problems before

they affect critical business processes. Nagios has the capability of monitoring applications, services and the entire IT infrastructure.

- NRPE is known as Nagios Remote Plugin Executor. It is designed to execute plugins on remote systems. On Windows hosts, the equivalent remote agent is NSClient++, which listens for Nagios requests and executes local checks (disk, CPU, memory, services, etc.).
- NSClient++ runs as a Windows service on the remote host and waits for requests from the Nagios server (usually over the NRPE protocol, which NSClient++ also supports). When the Nagios server needs to check a resource on that host, it sends a command; NSClient++ executes the check locally and returns the result to Nagios.

Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

Implementation:

1 - Pre-requisite: Install Docker Desktop

Download and install Docker Desktop for Windows and enable the WSL2 based engine. Verify the installation from PowerShell / Command Prompt

```
PS C:\Users\vishal> docker --version
PS C:\Users\vishal> docker version
```

Also make sure the WSL2 feature is enabled:

```
PS C:\Users\vishal> wsl --install
PS C:\Users\vishal> wsl --set-default-version 2
```

```
C:\Users\ANANYA>wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.

C:\Users\ANANYA>docker --version
Docker version 29.7.2, build a7dcaa6
```

2 - Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image and run it as a container, mapping the web port to the host.

```
PS C:\Users\vishal> docker pull jasonrivers/nagios:latest

PS C:\Users\vishal> docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
```

```
C:\Users\ANANYA>docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
6fe99ef1bed3: Pull complete
0d59a81a5608: Pull complete
dc3a88012a3e: Pull complete
f2f9a1b3ce48: Pull complete
d003d4303177: Pull complete
c42073aaf047: Pull complete
1b9771f55a12: Pull complete
a6b90b5bbb15: Pull complete
2908452ff193: Pull complete
1413dee23aae: Pull complete
59215b8e47cf: Pull complete
a80db686d641: Pull complete
fdbaa15b8986: Pull complete
e4dc1d4e010c: Pull complete
a7bd6dce6a77: Pull complete
a03720c2c16b: Pull complete
37f75c8146e2: Pull complete
50ea270d34e9: Pull complete
254354b70ac0: Pull complete
f47c3af6bf44: Pull complete
9288c4992679: Pull complete
c5701b5e1540: Pull complete
748b632e75e7: Pull complete
7edde1931fd2: Pull complete
e68fcda12c0c: Pull complete
80b6ab7aef5d: Pull complete
7a7447982198: Pull complete
54b9e993e315: Pull complete
cb259a83ac3d: Pull complete
Digest: sha256:fee9cd6a2933e790dde613d70c3a603f96978d65fa6bbc3d617fce807e00f2e2
Status: Downloaded newer image for jasonrivers/nagios:latest
docker.io/jasonrivers/nagios:latest
```

```
C:\Users\ANANYA>docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
6d669ceebea3a96fe5cd1f3d8cf10c056629109c24a5fdce368f1131503af01e
```

Verify the container is up and running:

```
PS C:\Users\vishal> docker ps
```

```
C:\Users\ANANYA>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS                                   NAMES
6d669ceebea3   jasonrivers/nagios:latest   "/usr/local/bin/star…"   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nagios4
```

3 - Set the Apache/Nagios Admin Password

Set (or reset) the password for the nagiosadmin user inside the running container:

```
PS C:\Users\vishal> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

Enter and confirm the password when prompted (e.g. 123456).

```
C:\Users\ANANYA>docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```

4 - Access Nagios Web Interface

Access the Nagios web interface from a browser on the Windows host:

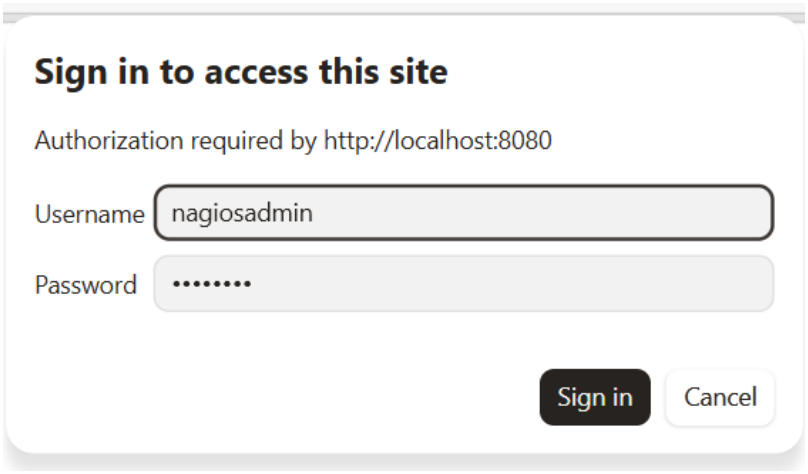
```
http://localhost:8080/nagios/
```

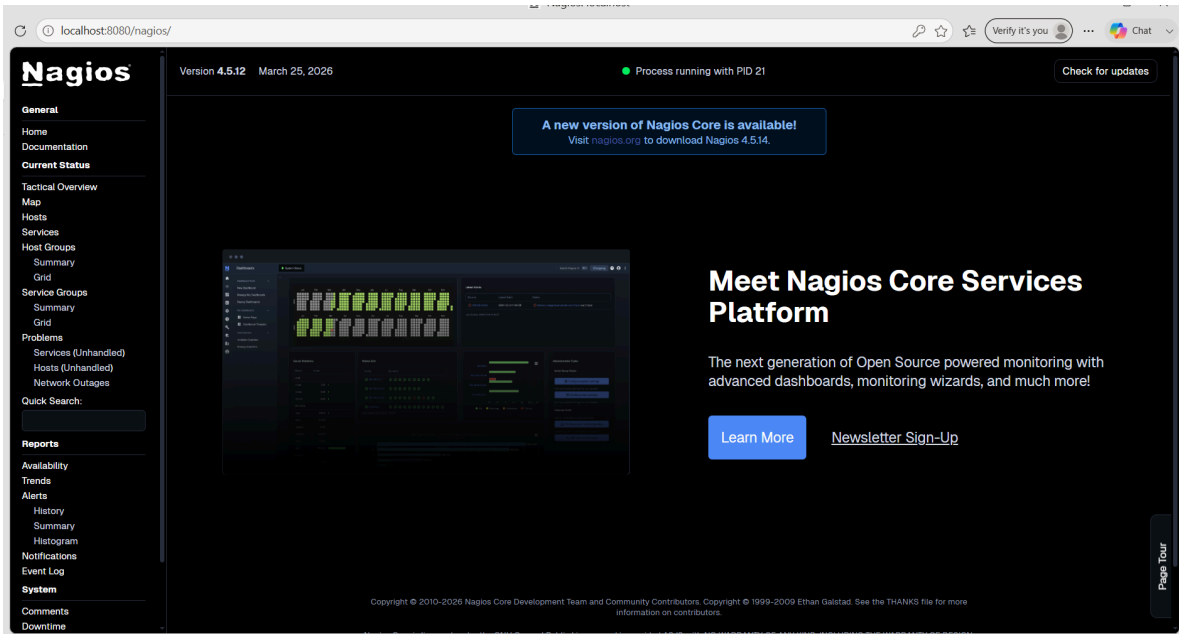
Prompting for Authentication Password -

username: nagiosadmin

Password: 12345678

```
C:\Users\ANANYA>start http://localhost:8080/nagios/
```





5 - Install NSClient++ on the Windows Remote Host

On the Windows machine you want to monitor, download and silently install NSClient++ (the Windows equivalent of the NRPE agent):

```
PS C:\Users\vishal> Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -OutFile "NSCP.msi"
PS C:\Users\vishal> Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```

```
C:\WINDOWS\system32>curl -L -o "%TEMP%\NSCP.msi" "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100     9  100     9    0     0      8      0  0:00:01  0:00:01 --:--:--     0

C:\WINDOWS\system32>msiexec.exe /i "%TEMP%\NSCP.msi" ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn /norestart
```

```
C:\WINDOWS\system32>powershell -Command "Invoke-WebRequest -Uri 'https://github.com/mickem/nscp/releases/download/0.5.2.35/NSCP-0.5.2.35-x64.msi' -OutFile 'D:\Downloads\NSCP.msi'"

C:\WINDOWS\system32>msiexec.exe /i "D:\Downloads\NSCP.msi" ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn /norestart

C:\WINDOWS\system32>sc query nscp

SERVICE_NAME: nscp
        TYPE               : 10  WIN32_OWN_PROCESS
        STATE              : 4  RUNNING
                               (STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE    : 0  (0x0)
        SERVICE_EXIT_CODE  : 0  (0x0)
        CHECKPOINT         : 0x0
        WAIT_HINT          : 0x0
```

6 - Configure NSClient++

Edit the NSClient++ configuration file and add the Nagios server's IP address to the allowed hosts list:

```
PS C:\Program Files\NSClient++> notepad nsclient.ini

allowed hosts=127.0.0.1,192.168.1.100
```

Where 192.168.1.100 is your Nagios server (Docker host) IP address.

```
C:\WINDOWS\system32>notepad "C:\Program Files\NSClient++\nsclient.ini"
```

```
; in flight - TODO
[/settings/default]

; Undocumented key
password = bLdF9qAPz0rssa3G

; Undocumented key
allowed hosts = 127.0.0.1, 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16
```

After saving the file, restart the NSClient++ service so the change takes effect:

```
PS C:\Users\vishal> Restart-Service nscp
```

```
C:\WINDOWS\system32>net stop nscp
The NSClient++ Monitoring Agent service is stopping.
The NSClient++ Monitoring Agent service was stopped successfully.


C:\WINDOWS\system32>net start nscp

The NSClient++ Monitoring Agent service was started successfully.
```

7 - Verify Connectivity from Nagios

From the Nagios server (inside the Docker container), run the NRPE check against the Windows remote host to confirm connectivity. Here 192.168.1.50 is the remote Windows host's IP address.

```
PS C:\Users\vishal> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.1.50
NRPE v2.15
```

```
C:\WINDOWS\system32>docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H host.docker.internal -n
I (0.5.2.35 2018-01-28) seem to be doing fine...

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```

8 - Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. We also define a service that continuously checks whether the host is reachable (PING check).

```
PS C:\Users\vishal> docker exec -it nagios4 sh -c "vi /opt/nagios/etc/servers/MyWindowsHost001.cfg"
define host {
   use             windows-server
   host_name       Windows_Host_001
   alias           Windows Host 001
   address         192.168.1.50
   register        1
}
define service{
   host_name            Windows_Host_001
```

```
    service_description    PING
    check_command          check_ping!100.0,20%!500.0,60%
    max_check_attempts     2
    check_interval         2
    retry_interval         2
    check_period           24x7
    check_freshness        1
    contact_groups         admins
    notification_interval  2
    notification_period    24x7
    notifications_enabled  1
    register               1
}
```

```
C:\WINDOWS\system32>docker exec -i nagios4 sh -c "cat << 'EOF' > /opt/nagios/etc/servers/MyWindowsHost001.cfg
define host {
    use             windows-server
    host_name       Windows_Host_001
    alias           Windows Host 001
    address         host.docker.internal
    register        1
}

define service {
    host_name               Windows_Host_001
    service_description     PING
    check_command           check_ping!100.0,20%!500.0,60%
    max_check_attempts      2
    check_interval          2
    retry_interval          2
    check_period            24x7
    check_freshness         1
    contact_groups          admins
    notification_interval   2
    notification_period     24x7
    notifications_enabled   1
    register                1
}
EOF"
```

Verify the configuration file for errors, then restart the Nagios container so the changes take effect:

```
PS C:\Users\vishal> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

PS C:\Users\vishal> docker restart nagios4
```

```
C:\WINDOWS\system32>docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
        Checked 8 services.
        Checked 2 hosts.
        Checked 2 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 2 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

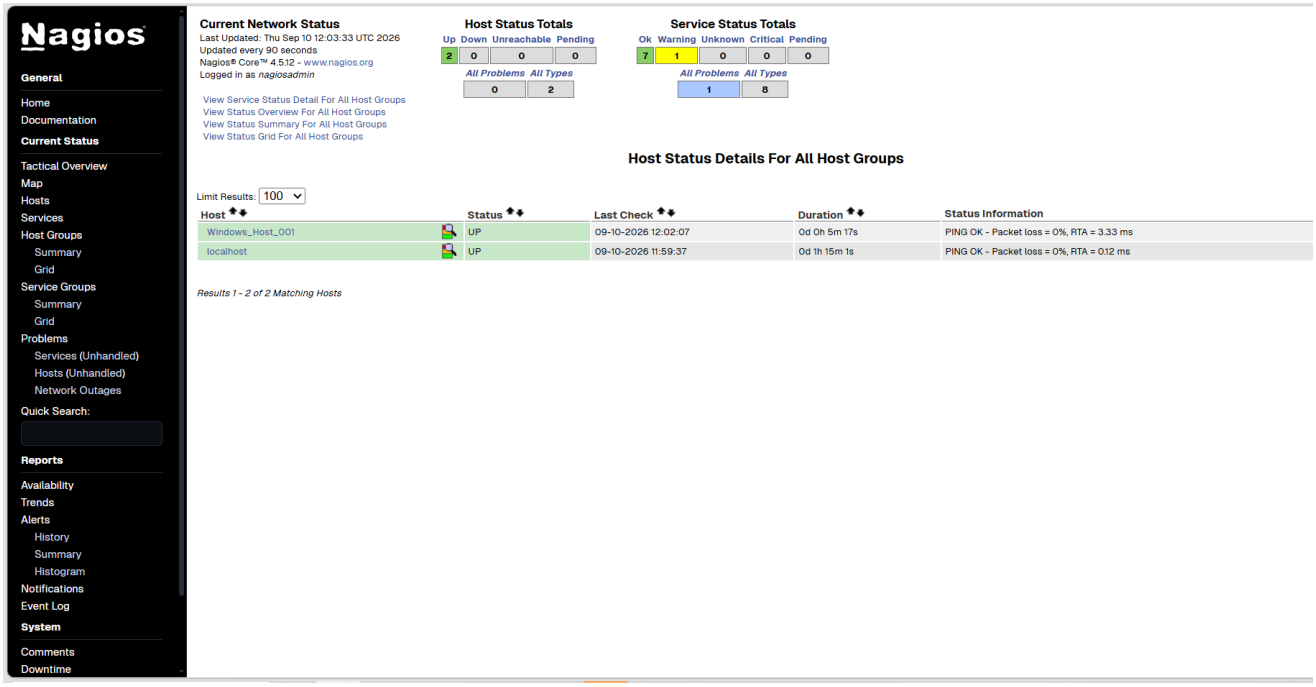
Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
```

```
C:\WINDOWS\system32>docker restart nagios4
nagios4
```

9 - Check Host in Nagios Web Interface

Open the Nagios web interface (http://localhost:8080/nagios/) and check that the new Windows host has been added and is being monitored under Current Status -> Hosts.



Conclusion:

Deploying Nagios Core within a Docker container to monitor a Windows host demonstrated the end-to-end mechanics of centralized infrastructure observability and agent-based telemetry. I learned how containerized environments communicate with host operating systems across virtualized subnets, as well as how to configure the NSClient++ daemon and bypass cryptographic cipher mismatches using targeted NRPE flags. The implementation provided practical insights into Nagios object architecture, particularly the structural dependencies required between host definitions, hostgroups, and active service checks like ICMP pinging. Resolving file system path mismatches and validating daemon configurations via pre-flight testing emphasized the importance of rigorous syntax verification prior to restarting production monitoring services. Overall, the practical work established a solid foundation in cross-platform network administration, host availability tracking, and real-time infrastructure diagnostics.